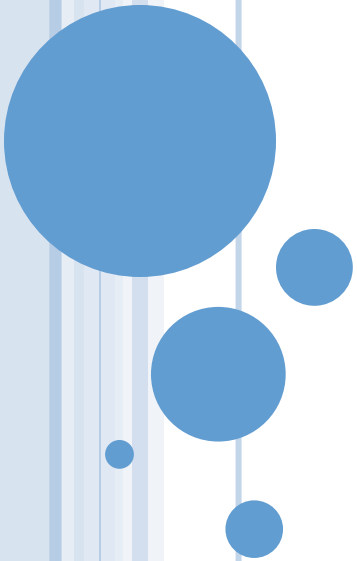


Worst Smells and their Worst Reasons

Davide Falessi, Ph.D.

Agenda

- Aim
- Definitions
- Practical Considerations
- Tool Support
  - Strengths
  - Limits
- Industrial Case Study
- Research Directions
- References

The Problem

- Most technical debt is seen as a compromise between quality and speed

- Some "worst smells" provide **no benefit whatsoever**—not even short-term speed—and should always be avoided.

The Economic Burden of Maintenance

- The majority of development effort over a project's lifetime and up to 80% of total costs are spent on maintenance.
- Code smells lead to technical debt, significantly decreasing maintainability.
- The Dilemma: Quality gates (automatic rejection of code commits) are often resisted because they are perceived to slow down "time to market".
- Goal of this Paper: Identify smells that can be blocked without any "time to market" penalty because they have no reason to exist.

Research Questions

- RQ1: Which smells are "worst" (have no justifiable reason to exist)?
- RQ2: Are worst smells meaningfully different from non-worst smells in terms of frequency, change-proneness, and severity?
- RQ3: What are the specific "worst reasons" for introducing these smells in the first place?

Methodology—The Developer Survey (RQ1)

- Participants: 71 developers recruited via the IEEE TCSE.
- Selection Process:
  - Started with 314 SonarCloud smells.
  - Used a pedagogical exercise with 60 students to filter out smells with "obvious" good reasons.
  - 133 candidate smells were presented to professional developers in a survey.
- Validation: Table I shows that 47% of respondents had 11+ years of experience, ensuring high-quality feedback.

TABLE I
LEVEL OF EXPERIENCE OF THE SURVEY RESPONDENTS.

Level of Experience	Proportion
Less than 1 year	12%
1-2 years	8%
2-3 Years	10%
3-5 years	7%
6-10 years	17%
11+ years	47%

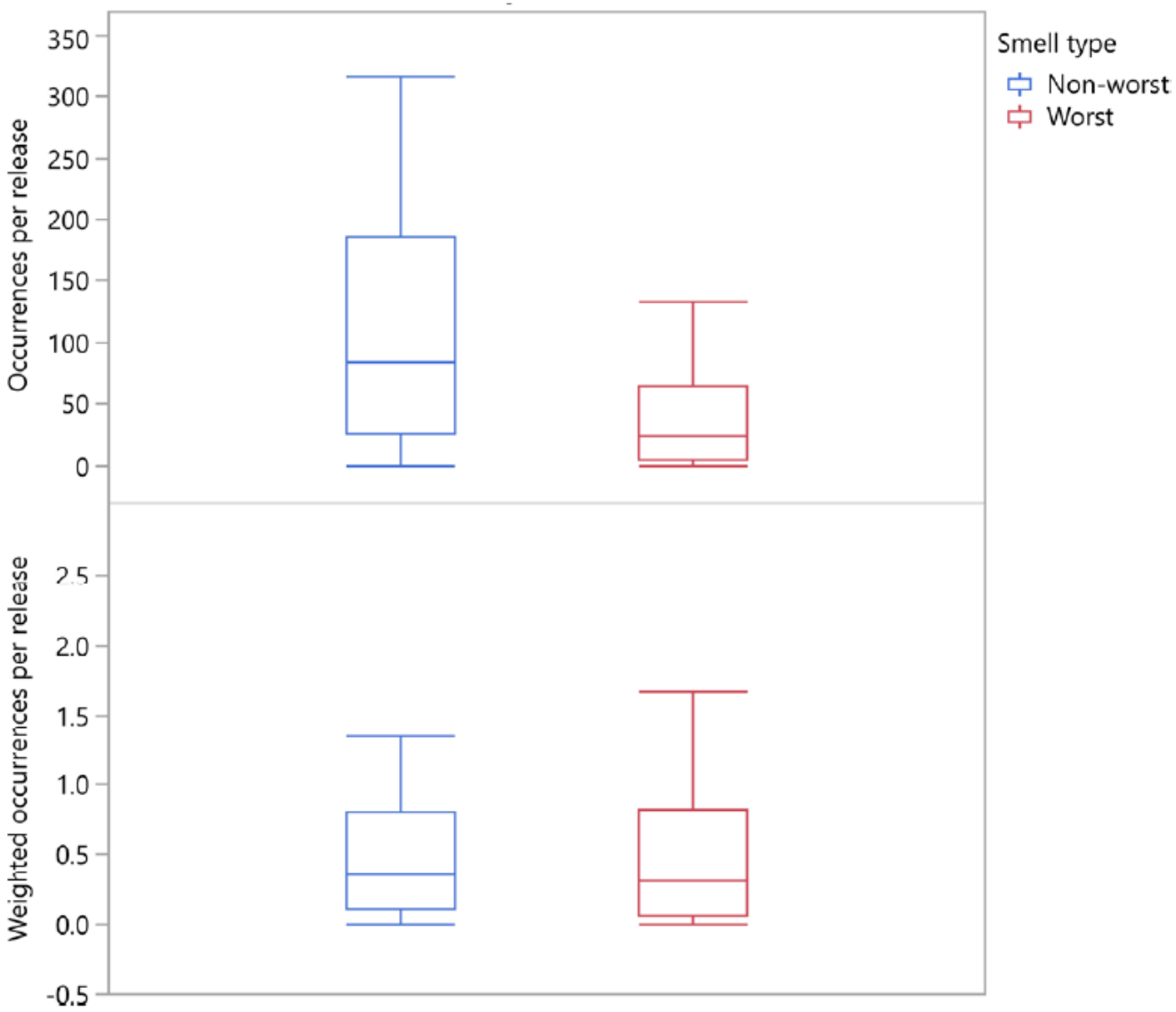
Results (RQ1)—The 80 Worst Smells

- Findings: 80 out of 314 smells (25.5%) were confirmed as "worst".
- Criteria: **A smell was "worst" if at least one respondent agreed there was no reason for it, and no respondent disagreed.**
- Example: Lazy naming (e.g., "Method1") or duplicated field names.
- **These are considered "mistakes of omission or commission" rather than strategic trade-offs.**

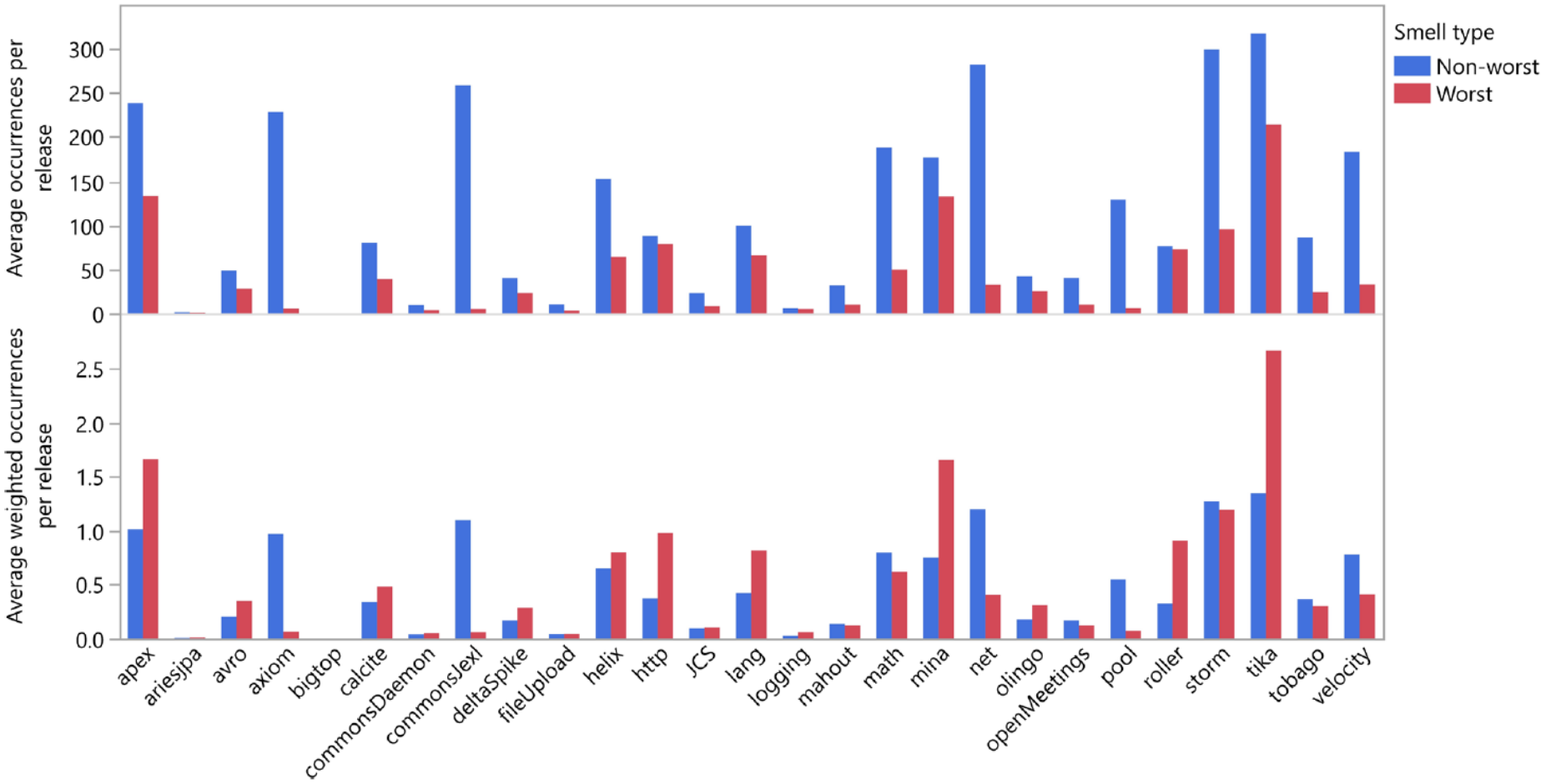
Methodology—Empirical Project Analysis (RQ2)

- Dataset: 27 large Apache open-source projects (Java/Maven based).
- Scope: 667 releases analyzed using a local SonarCloud server.
- Metrics Measured:
  - Frequency: Weighted occurrences to account for the different group sizes (80 worst vs. 234 non-worst).
  - Change-Proneness: Measured as the number of lines touched in the revision history.
  - Severity: Based on SonarCloud's scale (Info to Blocker).

Results (RQ2.1)—Are They More Frequent?



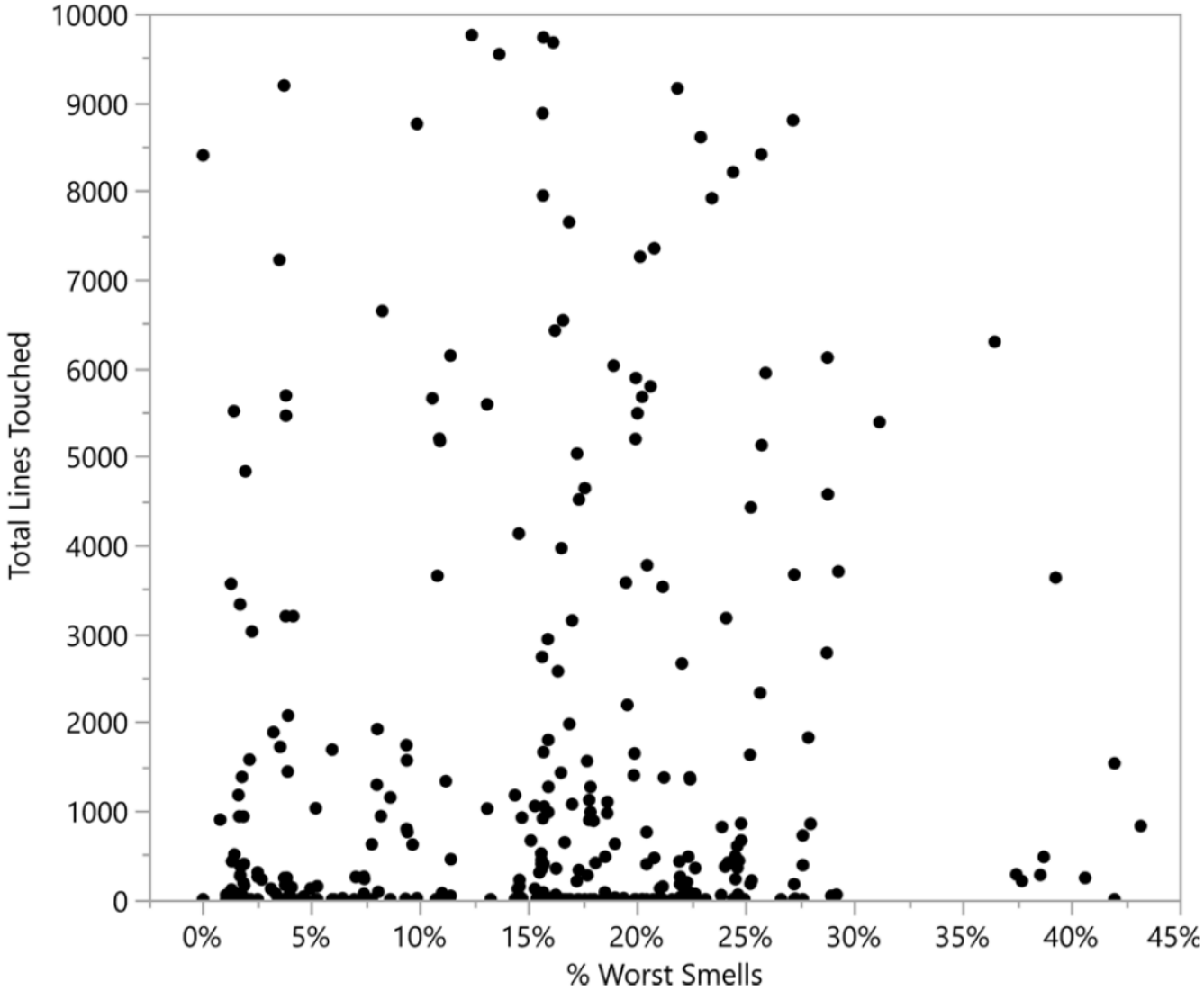
Results (RQ2.1)—Are They More Frequent?



Results (RQ2.1)—Are They More Frequent?

- In absolute terms, non-worst smells are more common.
- However, the likelihood of a developer violating a specific "worst" rule is statistically similar to a "non-worst" rule.

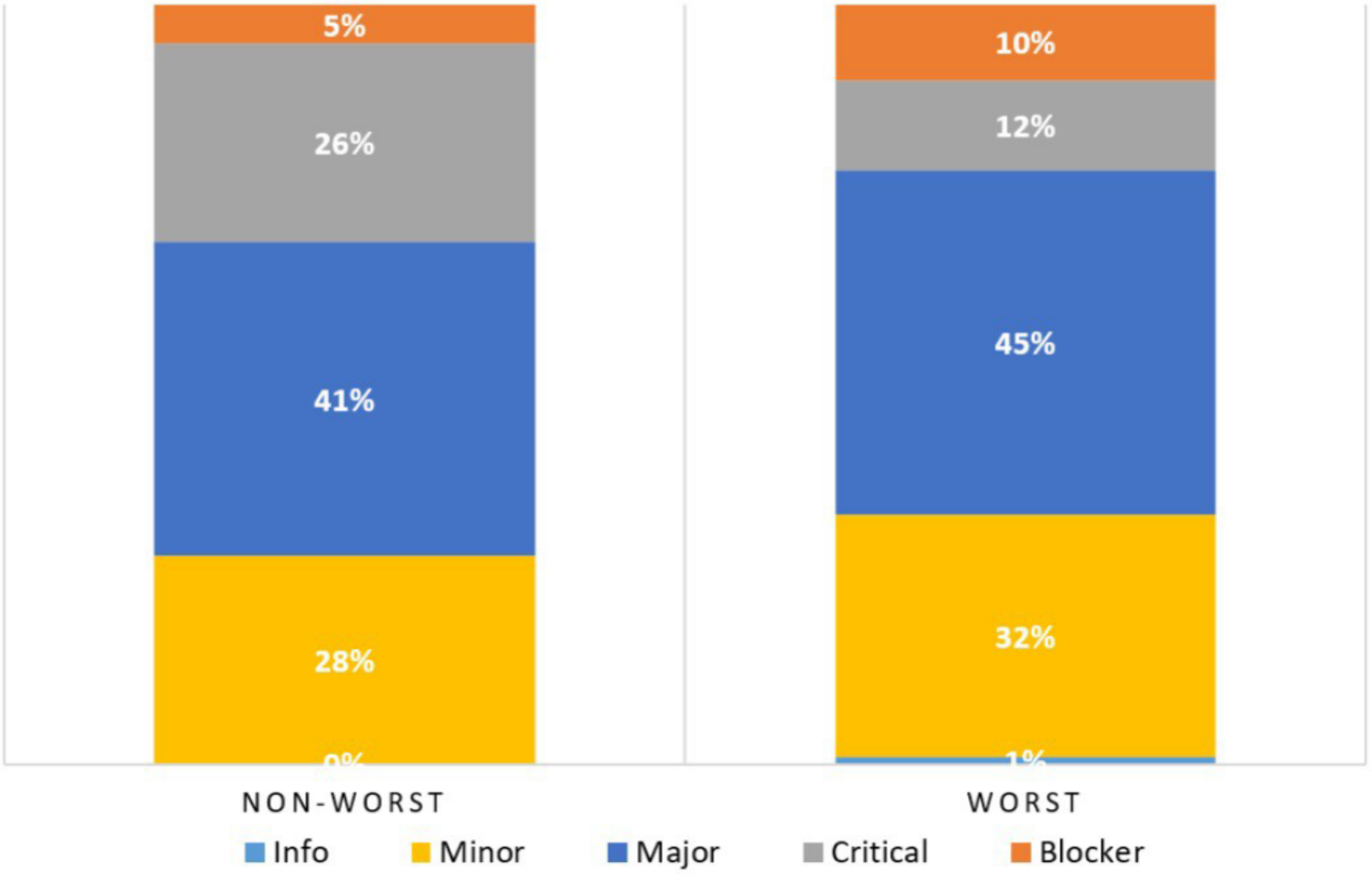
Results (RQ2.2)—Impact on Change-Proneness



Results (RQ2.2)—Impact on Change-Proneness

- Total occurrences of smells correlate with changed lines (debt impacts maintenance).
- However, the type of smell (worst vs. non-worst) does not change the amount of maintenance required.

Results (RQ2.3)—Severity Analysis



Results (RQ2.3)—Severity Analysis

- Worst smells generally have lower average severity (more "Minor" and "Major").
- Critically: Worst smells have a higher proportion of "Blocker" issues (10%) compared to non-worst smells (5%).

- Conclusion: Even if they seem minor on average, they frequently represent the most dangerous "Blocker" violations.

Results (RQ3)—Why Do We Introduce Them?

- Follow-up interviews with 5 senior developers.

Title	Definition	ID of Good Example	Title of Good Example
Lack of knowledge	A better solution is available but it is not known by the developer who, if they were aware of it, would have little or no reason to not use it. E.g., using modifiers in the correct order.	1124	Modifiers should be declared in the correct order.
Improper language transfer assumption	The logic behind the code would make sense in another language but not in the current (Java) language. E.g., implementing some sort of garbage collector in Java may be less optimal than the standard Java garbage collector but required in C/C++.	1113	The Object.finalize() method should not be overriden.
Committing preliminary code	The code was preliminary. and should have been removed before being committed to production. E.g., commented out code such as temporary code used in development as a prototype or for testing.	125	Sections of code should not be commented out.
Neglecting a simpler solution	The code would be okay if complemented by a lot of explanation. However a simpler solution is available which is more logical and hence does not require lengthy explanation. E.g.. missing an unconditional break in the final switch case. The absence of a break can be intentional (to be documented) or it can be a mistake.	128	”Switch cases should end with an unconditional ””break”” statement”.
Unintentional mistake	This is an unintentional mistake made because the developer was not thinking clearly. i.e.. an error in judgement rather than a lack of knowledge. E.g., using string concatenation in a loop rather than StringBuilder will degrade performance.	1643	Strings should not be concatenated using ’+’ in a loop.
Regression oversight	A piece of code makes no sense after a modification. E.g., an empty loop (which before was not empty).	108	Nested blocks of code should not be left empty.
Intentional mistake	The programmer was lazy and did not want to do things right and hence intentionally used a shortcut saving a few clicks. E.g., failing to use the ””@Override”” annotation to highlight that a method is being overridden does not save much time.”	1161	Override should be used on overriding and implementing methods

Discussion and Practical Application

- Actionable Advice: Developers should block these 80 smells in their quality gates immediately.
- Strategy by Reason:
  - Knowledge/Transfer: Solved by targeted education and training.
  - Regression/Preliminary Code: Solved by focused code reviews.
- Education: Reducing the 314 rules to 80 "worst" rules makes quality standards more manageable for students and new hires.

Conclusions

- 25.5% of code smells have no justifiable reason to exist.
- These smells are just as frequent and damaging as others, but offer no trade-off benefit.
- By targeting these smells, projects can improve quality without compromising agility.
- Final thought: "Technical debt" is a choice, but "worst smells" are just mistakes.